

Method Overloading (Without Var Args)
Command Line Argument
BLC & ELC class

```
package com.ravi.single_file_approach;

/*Single file approach
Take an input int value as a number
If the given number is 0 OR -ve then return -1
If the number is even return the square of the number
If the number is odd return the cube of the number
*/
//ELC class
public class Main
{
    void main()
    {
        int number = Integer.parseInt(IO.readLine("Enter a Number :"));
        int result = GetSquareAndCube.getSquareAndCube(number);
        IO.println("Result is :"+result);
    }
}

//BLC
class GetSquareAndCube
{
    public static int getSquareAndCube(int num)
    {
        if(num <=0)
        {
            return -1;
        }
        else if(num %2==0)
        {
            return num*num;
        }
        else
        {
            return num*num*num;
        }
    }
}
```

Types of Variable in Java :

* Based on the **data type**, We have **only two types** of variable in java :

- 1) Primitive Variables
- 2) Reference Variable

1) Primitive Variables

* If we declare a variable with primitive data type like byte, short, int, long, float, double, char and boolean then it is called primitive variables.

Example :
int x = 90; //primitive variables
boolean isEmpty = false; //primitive variables

* We cannot assign null literal on these primitive variables.

Example :
int x = null; //Invalid

* We cannot **call** any **method or field (variable)** by using primitive variable.

Example :
int x = 90;
x.m1(); //Invalid

2) Reference Variables :

* If we declare any **variable with reference data type** like class, array, record, interface, enum and so on **then it is called** reference variable.

Example :
Student s; //s is a reference variable
Scanner sc; //sc is a reference variable

* We can assign null literal on reference variable.

Example :
Student s = null;

* We can call non static field & non static method by using reference variable.

Example :
String str = "java";
IO.println(str.toUpperCase());

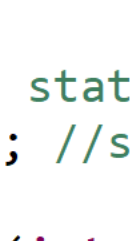
* Based on the **declaration position (Inside the method OR outside of the method)**, further variables are classified into 4 categories :

- 1) Static Field OR Class Variable
- 2) Non Static Field OR Instance Variable

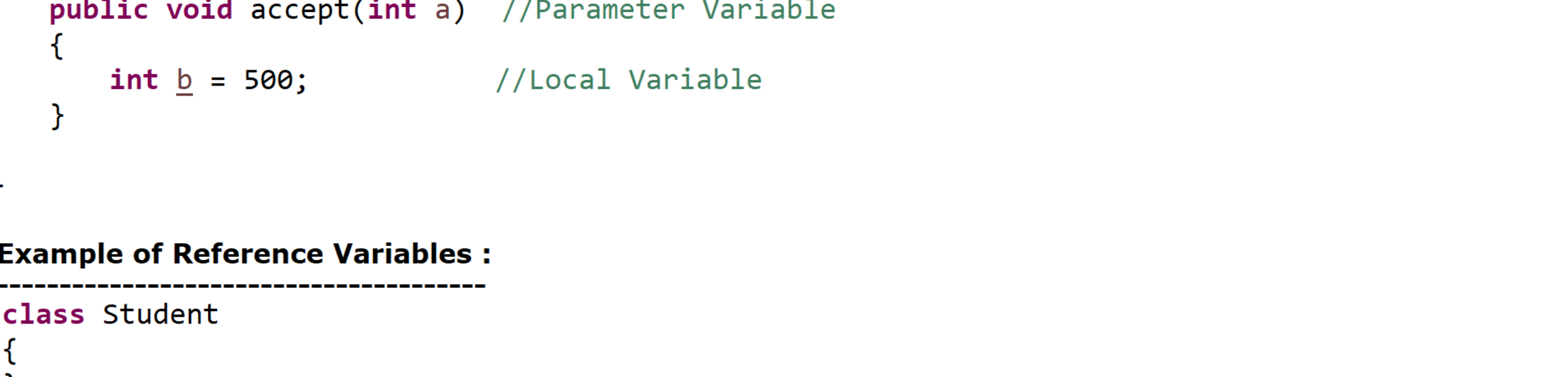


Class Level Variables

- 3) Local Variable
- 4) Parameter Variable



Method Level Variables



Example of Primitive Variables :

```
class Demo
{
    int x = 100; //Non static field
    static int y = 200; //static field

    public void accept(int a) //Parameter Variable
    {
        int b = 500; //Local Variable
    }
}
```

Example of Reference Variables :

```
class Student
{
}

class Demo
{
    Student s; //non static reference field
    static String str; //static reference field

    public void accept(Student s1) //s1 is a parameter reference variable
    {
        Student s2 = new Student(); //s2 is a local reference variable
    }
}
```

What is Object Oriented Programming (OOP)

What is an Object in Java ?

* An Object is a **physical entity which exist in the real word**.

Example :
Car, Fan, Remote, Mobile, Student e.t.c

* An object has 3 characteristics

- a) Id (Identification) of the object (Name of the object)
- b) State (Attribute OR Properties OR **non static field**) of the object
- c) Behaviour (Functionality OR non static methods) of the object

How we can create an object in java

* In order to create an object in java using new keyword, we need **class name**

Example :
public class Student
{
}



* Object Oriented Programming is a technique to design and develop java programs using **class and object**.

* In Object Oriented Programming, We always concentrate on objects rather than methods OR function.

* As a developer, If we are able to write java programs on real life object then we are Object Oriented Programmer and this technique is known as "Object Oriented Programming".

Advantages of OOPs:

- 1) Modularity (Dividing the bigger task into smaller task)
- 2) Reusability (We can reuse the component (classes and methods) so many times)
- 3) Flexibility (Easy to maintain [By using **interface**])

Features of OOPs:

- 1) Class
- 2) Object
- 3) Abstraction
- 4) Encapsulation
- 5) Inheritance
- 6) Polymorphism

What is a class ?

* A class is a **model OR prototype OR template OR blueprint** for creating an object.

Example :

